Assignment3

November 3, 2026

1 SYDE572 Assignment 3

1.1 Principal Component Analysis (PCA)

Name: Shiva Arya
Course: SYDE572
Assignment 3

GitHub link =

1.2 Part 1 - PCA by Hand

In this part, PCA is applied manually to a dataset containing five examples with two features each. The data are first centered, then the first principal component is identified. Reconstruction errors are also calculated for the points ((3,3)) and ((3,4)).

1.2.1 1.1 Dataset and Centering

The dataset is:

$$(-2,-1),\ (-1,0),\ (0,1),\ (1,2),\ (2,3)$$

The mean of the first feature is:

$$\bar{x}_1 = \frac{-2-1+0+1+2}{5} = 0$$

The mean of the second feature is:

$$\bar{x}_2 = \frac{-1+0+1+2+3}{5} = 1$$

Therefore, the mean vector is:

$$\mu = (0,1)$$

After subtracting the mean from each point, the centered dataset is:

$$(-2,-2),\ (-1,-1),\ (0,0),\ (1,1),\ (2,2)$$

1.2.2 1.2 First Principal Component

The centered data points all lie along the direction:

$$(1, 1)$$

The length of this direction vector is:

$$\sqrt{1^2 + 1^2} = \sqrt{2}$$

Therefore, the normalized first principal component is:

$$w_1 = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}} \right)$$

Approximately:

$$w_1 = (0.7071,\ 0.7071)$$

The opposite direction,

$$(-0.7071,\ -0.7071)$$

represents the same principal component axis.

1.2.3 1.3 Reconstruction Error for ((3,3))

The point is:

$$x = (3, 3)$$

The mean of the dataset is:

$$\mu = (0, 1)$$

First, center the point:

$$x_c = x - \mu$$

$$x_c = (3, 3) - (0, 1) = (3, 2)$$

The first principal component is:

$$w_1 = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$$

The projection coefficient is:

$$z = w_1^T x_c$$

$$z = \frac{1}{\sqrt{2}}(3) + \frac{1}{\sqrt{2}}(2) = \frac{5}{\sqrt{2}}$$

The reconstructed centered point is:

$$\hat{x}_c = z w_1$$

$$\hat{x}_c = \frac{5}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right) = (2.5, 2.5)$$

Adding the mean back gives:

$$\hat{x} = (2.5, 2.5) + (0, 1) = (2.5, 3.5)$$

The L2 reconstruction error is:

$$\|x - \hat{x}\|_2 = \sqrt{(3 - 2.5)^2 + (3 - 3.5)^2}$$

$$= \sqrt{0.25 + 0.25} = \sqrt{0.5} = \frac{1}{\sqrt{2}}$$

Therefore:

$$\text{Reconstruction error} \approx 0.7071$$

1.2.4 1.4 Reconstruction Error for ((3,4))

The point is:

$$x = (3, 4)$$

The mean of the dataset is:

$$\mu = (0, 1)$$

First, center the point:

$$x_c = x - \mu$$

$$x_c = (3,4) - (0,1) = (3,3)$$

The centered point ((3,3)) lies exactly along the first principal component direction ((1,1)).

Therefore, projecting the point onto the first principal component does not change it:

$$\hat{x}_c = (3,3)$$

Adding the mean back gives:

$$\hat{x} = (3,3) + (0,1) = (3,4)$$

The L2 reconstruction error is:

$$\|x - \hat{x}\|_2 = \sqrt{(3-3)^2 + (4-4)^2} = 0$$

Therefore:

$$\text{Reconstruction error} = 0$$

This is because the point ((3,4)) lies exactly on the principal component line.

1.3 Part 2 - Data Visualization

The Iris dataset contains measurements from three different types of Iris flowers. Each flower has four features:

- sepal length
- sepal width
- petal length
- petal width

In this part, I will visualize the data using one, two, and three features. The aim is to see which features make it easier to separate the three flower classes.

[5]:
```
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import numpy as np

iris = load_iris()

X = iris.data
y = iris.target
```

```
feature_names = iris.feature_names
target_names = iris.target_names

print("Dataset shape:", X.shape)
print("Features:", feature_names)
print("Classes:", target_names)
```

```
Dataset shape: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)']
Classes: ['setosa' 'versicolor' 'virginica']
```

1.3.1 2.1 Visualization Using One Feature

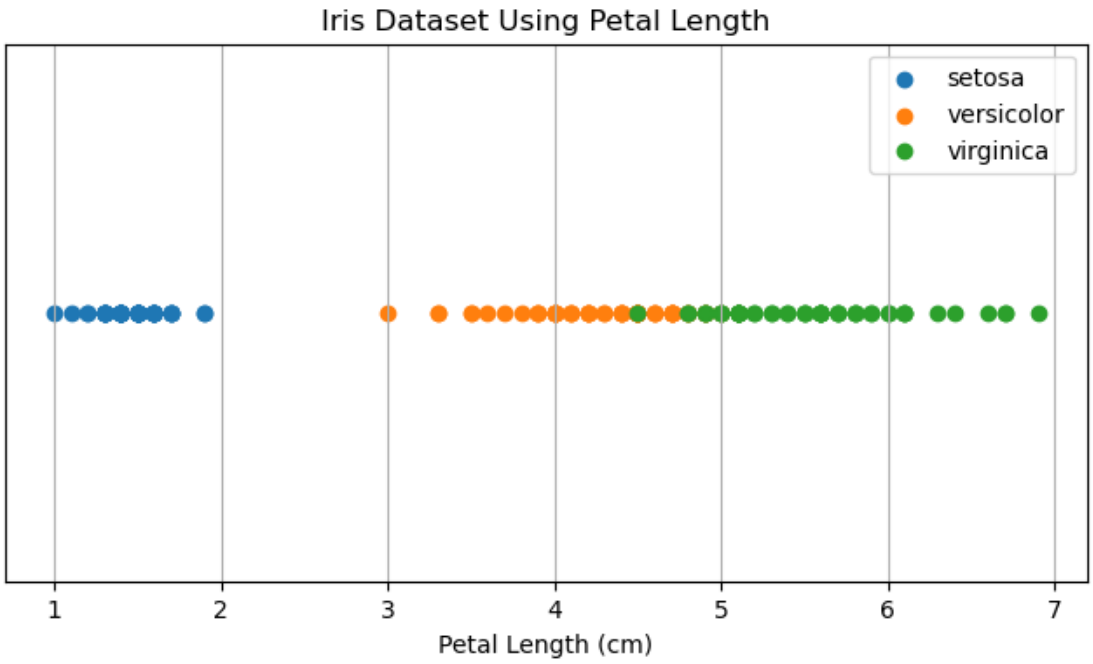
I first visualized the dataset using only petal length. This makes it possible to see how much the three flower classes can be separated using just one measurement.

[6]:
```
plt.figure(figsize=(8, 4))

for i, name in enumerate(target_names):
    plt.scatter(
        X[y == i, 2],
        np.zeros(np.sum(y == i)),
        label=name
    )

plt.xlabel("Petal Length (cm)")
plt.yticks([])
plt.title("Iris Dataset Using Petal Length")
plt.legend()
plt.grid()

plt.show()
```



Using only petal length, the Setosa class is clearly separated from the other two classes. Versicolor and Virginica overlap more, so petal length alone is not enough to separate all three classes perfectly.

1.3.2 2.2 Visualization Using Two Features

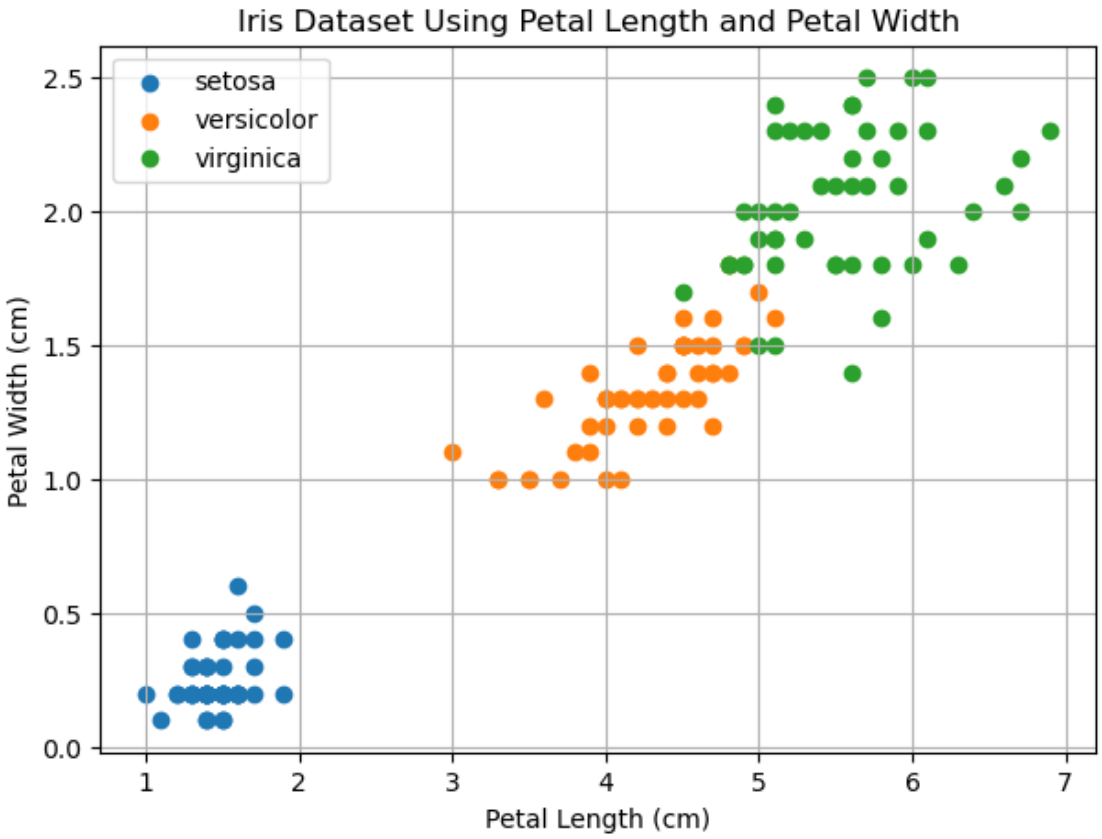
Next, I used petal length and petal width together. Using two features gives a clearer view of how the three flower classes are grouped.

[9]:
```
plt.figure(figsize=(7, 5))

for i, name in enumerate(target_names):
    plt.scatter(
        X[y == i, 2],
        X[y == i, 3],
        label=name
    )

plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.title("Iris Dataset Using Petal Length and Petal Width")
plt.legend()
plt.grid()

plt.show()
```



Using petal length and petal width gives a much clearer separation between the classes. Setosa is completely separated, while Versicolor and Virginica are also easier to distinguish compared with the one-feature plot, although there is still some overlap.

1.3.3 2.3 Visualization Using Three Features

Finally, I used sepal length, petal length, and petal width together. A 3D plot allows the relationship between three measurements to be viewed at the same time.

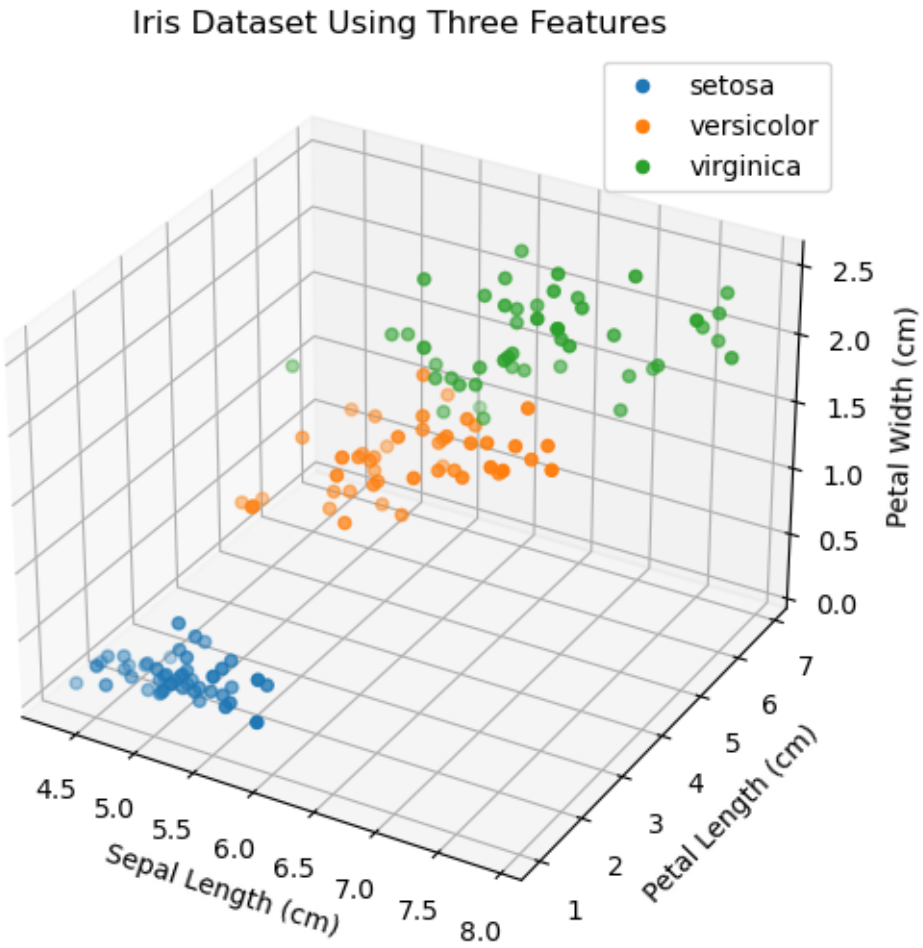
[10]:
```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

for i, name in enumerate(target_names):
    ax.scatter(
        X[y == i, 0],
        X[y == i, 2],
        X[y == i, 3],
        label=name
    )
```

```
ax.set_xlabel("Sepal Length (cm)")
ax.set_ylabel("Petal Length (cm)")
ax.set_zlabel("Petal Width (cm)")
ax.set_title("Iris Dataset Using Three Features")
ax.legend()

plt.show()
```



Using three features also shows clear grouping between the flower classes. Setosa remains very well separated, while Versicolor and Virginica are easier to distinguish than in the one-feature plot. However, there is still some overlap between these two classes.

1.3.4 2.4 Conclusion

From the three visualizations, petal length and petal width gave the clearest separation between the Iris classes. Setosa was easy to separate in all of the plots, while Versicolor and Virginica had some overlap. Using two or three features made the class structure easier to see than using only

one feature.

1.4 Part 3 - Principal Component Analysis

In this part, PCA is applied to the Iris dataset. PCA creates new axes, called principal components, that capture as much variation in the data as possible.

I will visualize the dataset using the first principal component, the first two principal components, and the first three principal components.

[11]:
```
from sklearn.decomposition import PCA

pca = PCA(n_components=4)
X_pca = pca.fit_transform(X)

print("PCA axes:")
print(pca.components_)
```

```
PCA axes:
[[ 0.36138659 -0.08452251  0.85667061  0.3582892 ]
 [ 0.65658877  0.73016143 -0.17337266 -0.07548102]
 [-0.58202985  0.59791083  0.07623608  0.54583143]
 [-0.31548719  0.3197231   0.47983899 -0.75365743]]
```

1.4.1 3.1 Visualization Using the First Principal Component

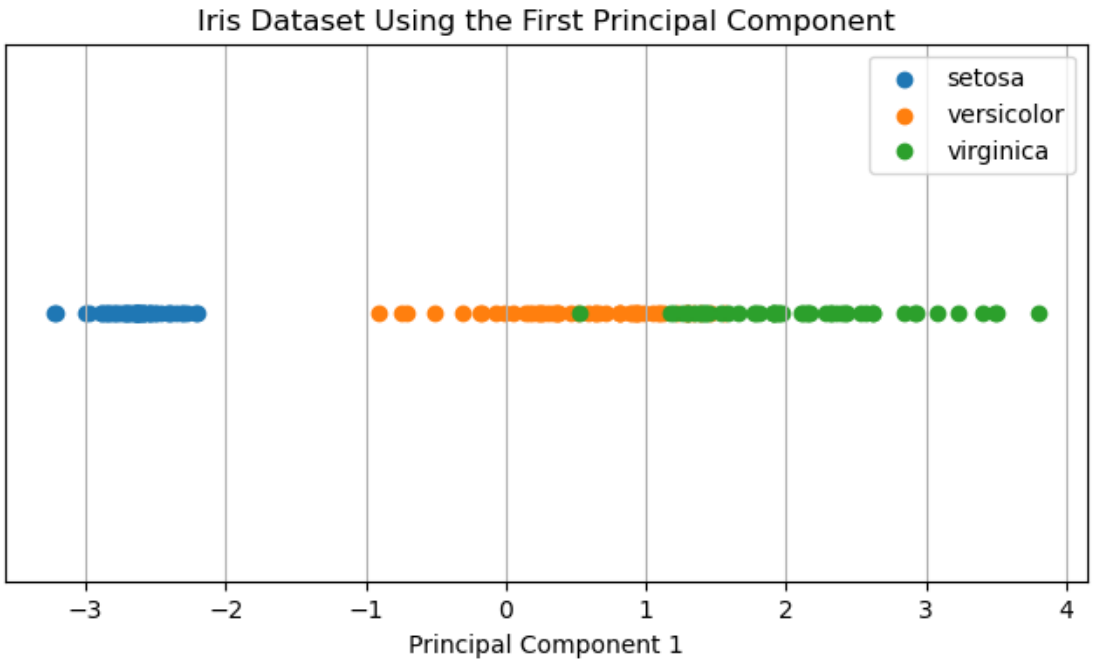
First, I visualized the Iris dataset using only the first principal component. This shows how well a single PCA direction can separate the three flower classes.

[12]:
```
plt.figure(figsize=(8, 4))

for i, name in enumerate(target_names):
    plt.scatter(
        X_pca[y == i, 0],
        np.zeros(np.sum(y == i)),
        label=name
    )

plt.xlabel("Principal Component 1")
plt.yticks([])
plt.title("Iris Dataset Using the First Principal Component")
plt.legend()
plt.grid()

plt.show()
```



The first principal component already separates Setosa very clearly from the other two classes. Versicolor and Virginica are also partly separated, but there is still some overlap between them.

1.4.2 3.2 Visualization Using the First Two Principal Components

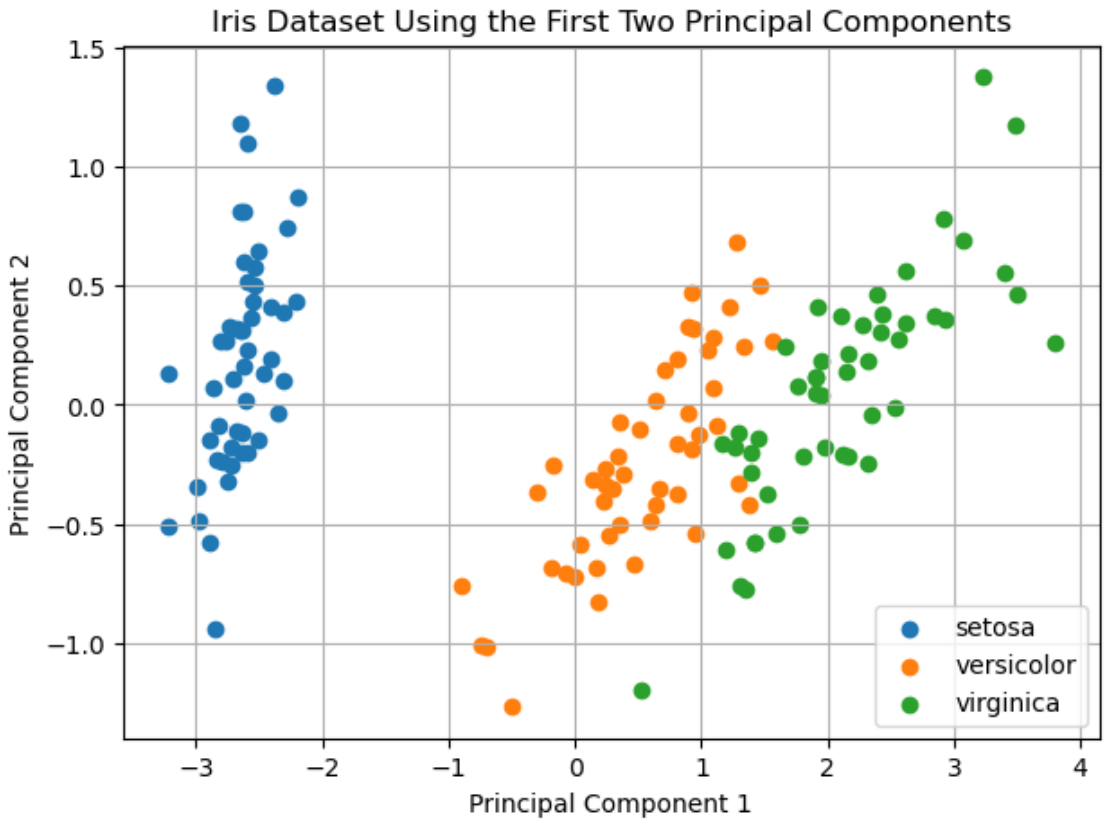
Next, I used the first two principal components together. This gives a two-dimensional view of the data while keeping the directions that explain the most variation.

[13]:
```
plt.figure(figsize=(7, 5))

for i, name in enumerate(target_names):
    plt.scatter(
        X_pca[y == i, 0],
        X_pca[y == i, 1],
        label=name
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Iris Dataset Using the First Two Principal Components")
plt.legend()
plt.grid()

plt.show()
```



Using the first two principal components gives a clearer view of the class structure. Setosa is completely separated from the other two classes, while Versicolor and Virginica are also more clearly grouped, although there is still some overlap between them.

1.4.3 3.3 Visualization Using the First Three Principal Components

Finally, I used the first three principal components to create a 3D visualization. This keeps more information from the original dataset while still reducing the data from four dimensions to three.

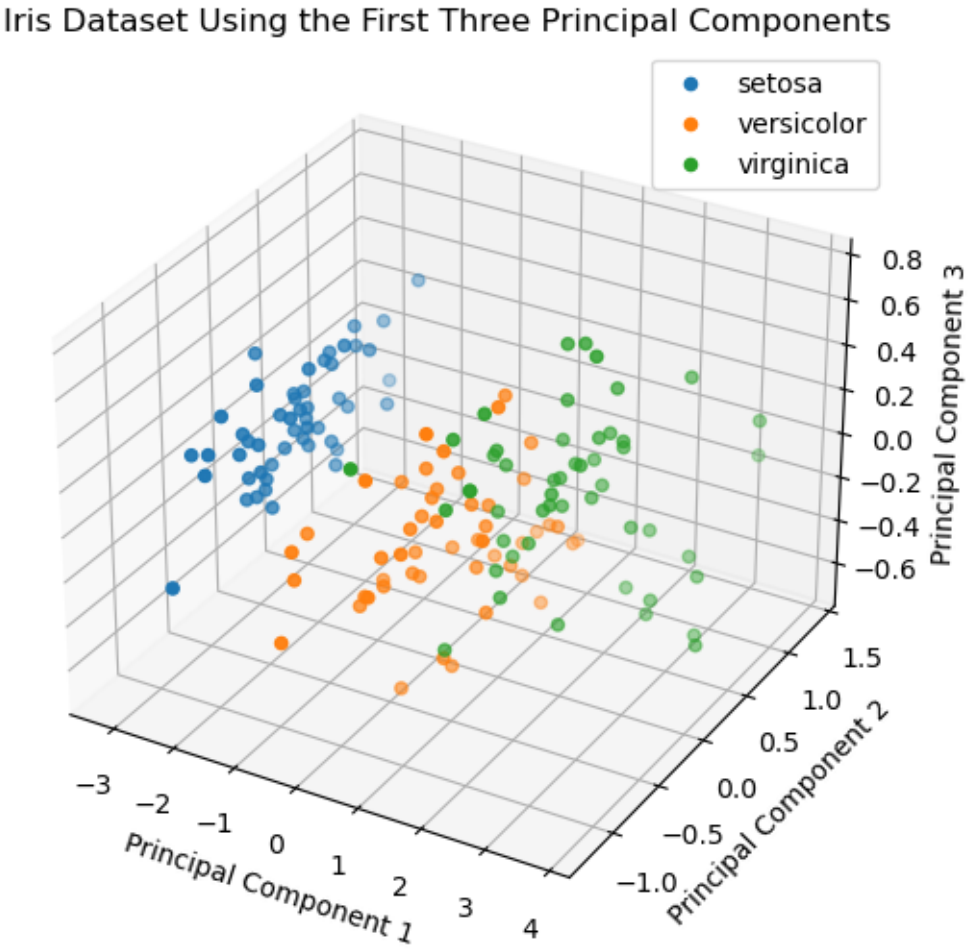
[14]:
```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

for i, name in enumerate(target_names):
    ax.scatter(
        X_pca[y == i, 0],
        X_pca[y == i, 1],
        X_pca[y == i, 2],
        label=name
    )

ax.set_xlabel("Principal Component 1")
```

```
ax.set_ylabel("Principal Component 2")
ax.set_zlabel("Principal Component 3")
ax.set_title("Iris Dataset Using the First Three Principal Components")
ax.legend()

plt.show()
```



The three-dimensional PCA plot also shows clear grouping between the classes. Setosa remains very well separated, while Versicolor and Virginica are mostly separated but still have a small amount of overlap.

1.4.4 3.4 PCA Axis Vectors

The PCA axes are the directions found by PCA in the original four-feature space. Each principal component is represented by four values, one for each original Iris feature.

[15]:
```
for i, component in enumerate(pca.components_):
    print(f"PC{i+1}: {component}")
```

```
PC1: [ 0.36138659 -0.08452251  0.85667061  0.3582892 ]
PC2: [ 0.65658877  0.73016143 -0.17337266 -0.07548102]
PC3: [-0.58202985  0.59791083  0.07623608  0.54583143]
PC4: [-0.31548719  0.3197231   0.47983899 -0.75365743]
```

Each principal component is a combination of the original four features. The first principal component represents the direction with the largest variation in the dataset, while the later components describe smaller remaining variations.

1.5 Part 4 - Data Compression

PCA can also be used for data compression. By keeping only the principal components that explain most of the variation in the data, the number of dimensions can be reduced while still keeping most of the useful information.

In this part, I calculate how much variance is explained by the first two principal components and how many components are needed to explain at least 50% of the total variance.

[16]:
```
print("Explained variance ratio:")
print(pca.explained_variance_ratio_)

print("\nCumulative explained variance:")
print(np.cumsum(pca.explained_variance_ratio_))
```

```
Explained variance ratio:
[0.92461872 0.05306648 0.01710261 0.00521218]

Cumulative explained variance:
[0.92461872 0.97768521 0.99478782 1.        ]
```

[17]:
```
variance_2d = np.sum(pca.explained_variance_ratio_[:2])

print("Variance explained by first two PCs:", variance_2d)
print("Percentage:", variance_2d * 100)
```

```
Variance explained by first two PCs: 0.977685206318795
Percentage: 97.7685206318795
```

[18]:
```
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

num_components_50 = np.argmax(cumulative_variance >= 0.50) + 1

print("Number of PCs needed to explain at least 50% of variance:",
      num_components_50)
```

```
Number of PCs needed to explain at least 50% of variance: 1
```

1.5.1 4.1 Variance Explained by Two Principal Components

The first two principal components explain about **97.8%** of the total variance in the Iris dataset.

This means that reducing the original four-dimensional dataset to only two principal components keeps almost all of the variation in the data.

1.5.2 4.2 Number of Components Needed for 50% Variance

The first principal component alone explains more than 50% of the total variance.

Therefore, only **one principal component** is needed to explain at least 50% of the variance in the dataset.

1.6 Part 5 - Non-linear Components

PCA is a linear dimensionality reduction method, so it works best when the important structure in the data can be represented using straight directions.

ISOMAP is a non-linear dimensionality reduction method. Instead of only using straight-line distances between points, it tries to preserve distances along the structure of the data.

In this part, I use ISOMAP with 3 nearest neighbours and compare the result with PCA.

[21]:
```
import warnings
warnings.filterwarnings("ignore")

from sklearn.manifold import Isomap
import matplotlib.pyplot as plt

# Apply ISOMAP using 3 nearest neighbours
isomap = Isomap(n_neighbors=3, n_components=2)

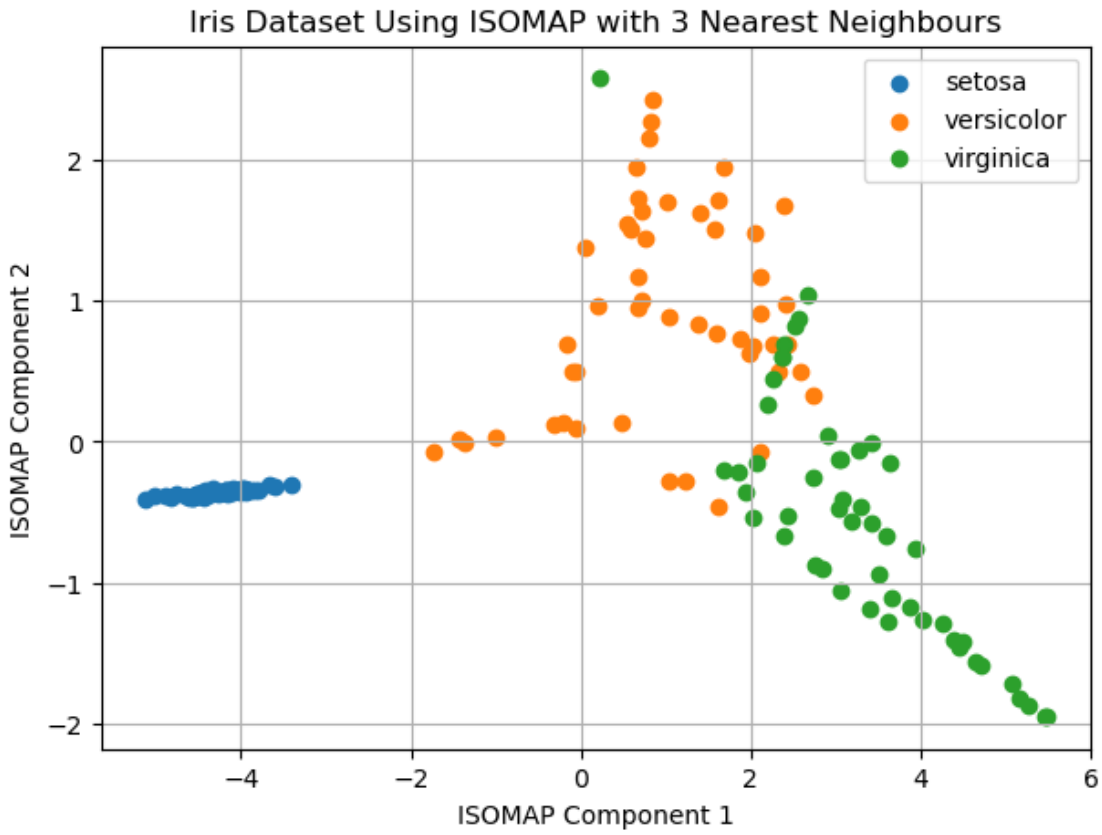
# Transform the original Iris data into 2 dimensions
X_isomap = isomap.fit_transform(X)

# Plot the result
plt.figure(figsize=(7, 5))

for i, name in enumerate(target_names):
    plt.scatter(
        X_isomap[y == i, 0],
        X_isomap[y == i, 1],
        label=name
    )

plt.xlabel("ISOMAP Component 1")
plt.ylabel("ISOMAP Component 2")
plt.title("Iris Dataset Using ISOMAP with 3 Nearest Neighbours")
plt.legend()
plt.grid()

plt.show()
```



1.6.1 5.1 Comparison with PCA

Both PCA and ISOMAP reduce the Iris dataset to fewer dimensions, but they do it in different ways.

PCA is a linear method, so it finds straight directions that explain the most variation in the data. ISOMAP is a non-linear method and tries to preserve the structure of the data by using distances between nearby points.

From the plots, both methods separate Setosa very clearly. Versicolor and Virginica still have some overlap in both cases.

For this dataset, PCA is preferable because it gives a clear separation while also providing explained-variance information, which makes it easier to measure how much information is kept after dimensionality reduction. ISOMAP can be useful when the data lies on a curved or non-linear structure, but for the Iris dataset the advantage is not very large.

1.7 Appendix - Handwritten PCA Calculations

The following pages contain my handwritten calculations for Part 1, including the centering of the data, calculation of the first principal component, and reconstruction errors for ((3,3)) and ((3,4)).

Q1

PC1 $= \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ $\quad \|w\| = 1 \quad \sqrt{w_1^2 + w_2^2} = 1$

$x_c = x - \mu$ centering the data

Data set $(-2,-1)\ (-1,0)\ (0,1)\ (1,2)\ (2,3)$

mean $\dfrac{-2-1+0+1+2}{5} = 0$ first feature

mean $\dfrac{-1+0+1+2+3}{5} = 1$ Second feature

$\mu = (0,1)$

Center the data

$(-2,-1) - (0,1) = (-2,-2)$

$(-1,0) - (0,1) = (-1,-1)$

$(0,1) - (0,1) = (0,0)$

$(1,2) - (0,1) = (1,1)$

$(2,3) - (0,1) = (2,2)$

$\longrightarrow$ in the direction $(1,1)$ $\quad \|w\| = 1$

length $= \sqrt{1+1} = \sqrt{2}$

$w_1 = \dfrac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \rightsquigarrow w = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$

Q2

Q3 $(3,4) - (0,1) = (3,3)$

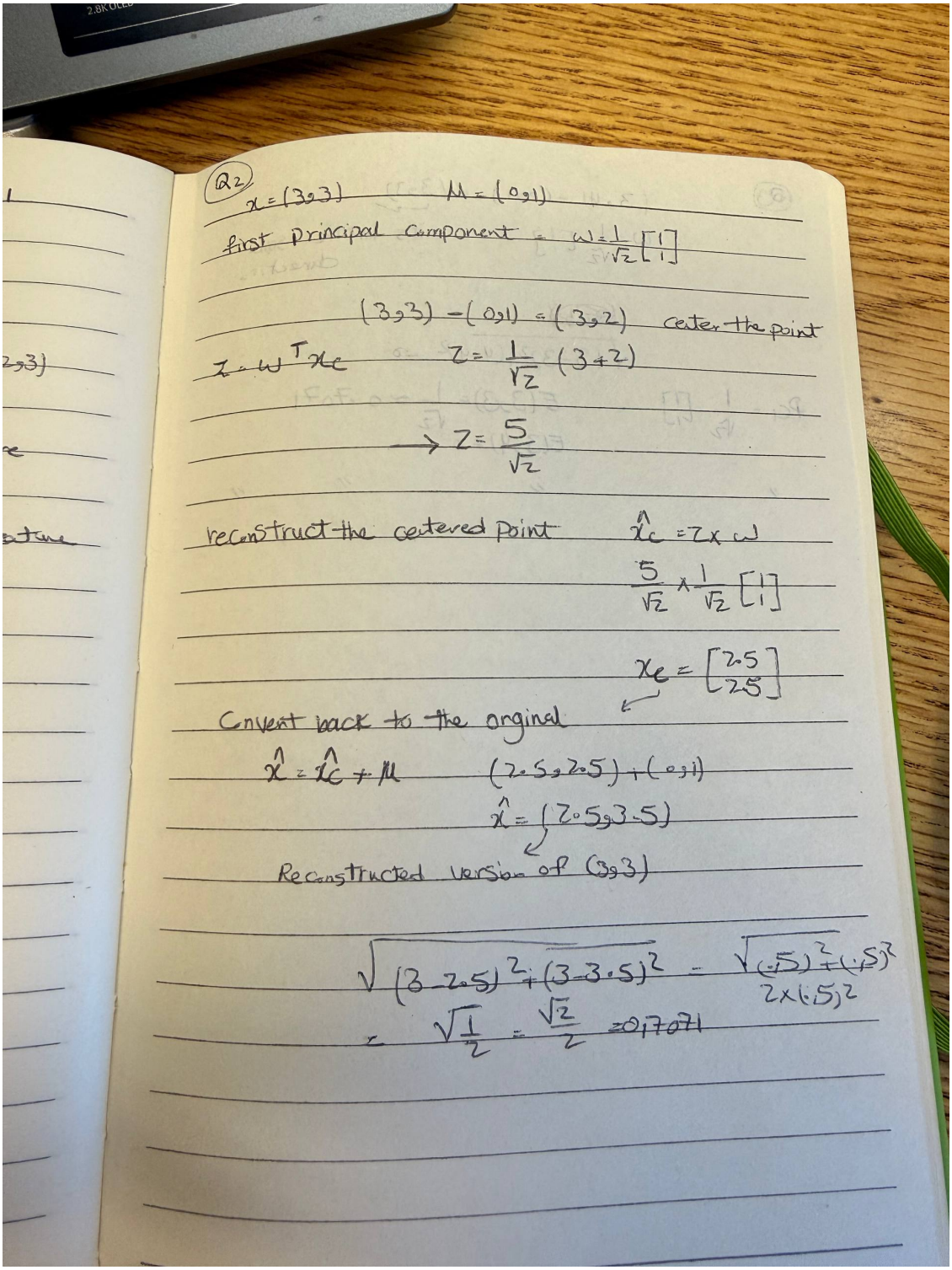
$w = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$ Its in the same direction

$\sqrt{(3-3)^2+(4-4)^2} = 0$

$PC_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$ $E(3,3) = \frac{1}{\sqrt{2}} \approx 0.7071$

$E(3,4) = 0$

" " " "


Q2
x = (3,3) μ = (0,1)
first principal component = w = 1/√2 [1 1]
(3,3) − (0,1) = (3,2) center the point
z = w^T x_c z = 1/√2 (3+2)
→ z = 5/√2
reconstruct the centered point x̂_c = z × w
5/√2 × 1/√2 [1 1]
x_c = [2.5 2.5]
Convert back to the orginal
x̂ = x̂_c + μ (2.5,2.5) + (0,1)
x̂ = (2.5,3.5)
Reconstructed version of (3,3)
√((3−2.5)² + (3−3.5)²) = √((.5)² + (.5)²) / 2×(.5)²
= √1/2 = √2/2 = 0,7071

[ ]: